Selection Is Not Self-Improvement

Dylan Wilson

Early draft · November 2026

Abstract

Patch-based self-improvement systems allow an agent to modify the code that will later modify the agent. This makes them look like concrete instances of recursive self-improvement: the optimizer is inside the edit surface, so the system can, in principle, get better at getting better. We analyze the published artifacts of DGM-H/HyperAgents across paper review, robot-walking reward design, and IMO-style proof tasks, together with the authors' transfer ablations and a pre-registered one-shot baseline. The results support a narrower mechanism. The systems improve, but the improvements come from feedback-driven patching of the task agent and from a one-time construction of a general agent harness. Continued meta-level self-modification does not appear to add content beyond the frozen base model's prior.

Three observations drive the conclusion. First, the authors' own held-out transfer ablation finds that a frozen evolved meta-agent matches a continuing self-editing meta-agent at matched budget: 0.688 versus 0.680 best accuracy within 50 generations. Second, meta-edits in the published runs are rare, mostly generic, and often not wired into the executing agents; reconstructing the final repositories reduces the live meta machinery to evaluation digests in three runs and a memory tool in one. Third, in a pre-registered comparison against one-shot proposals from the same frozen model, blinded judges can distinguish task-level loop edits from one-shot prior samples, but not meta-level edits: task AUC is 0.717, while meta AUC is 0.465, with 86% of loop meta interventions covered by 30 one-shot samples.

We also report a demoted but still informative extension to philosophy writing under LLM-judged reward. It reproduces the same qualitative pattern: early task-level gains, convergent harness motifs, high elicitation sensitivity, and no evidence that forced or prioritized meta-editing improves the improver. Because that setting is subjective and lower-powered, we treat it as corroborative rather than load-bearing.

The central distinction is between selection and self-improvement. Scalar evaluation can reject broken variants and retain useful task patches. It does not, by itself, attribute outcomes to meta-level design choices strongly enough to steer the improver. Claims of standing self-improvement should therefore be benchmarked against frozen-framework and one-shot-prior baselines. In the artifacts studied here, those baselines explain the observed meta-level gains.

1 Introduction

Patch-based recursive self-improvement is one of the clearest current proposals for self-improving AI. In the Darwin Gödel Machine and its hyper-agent successor DGM-H/HyperAgents, an agent is represented as a repository. Each generation, a meta-agent inspects the repository and evaluation artifacts, edits any part of the codebase, and submits the modified agent for evaluation. The resulting score determines whether the child enters the archive and can become a parent for later generations. Because the repository includes the task agent, the prompts, the tools, and the meta-agent itself, the optimizer is part of what can be optimized.

That architecture invites a strong interpretation: some of the gains come from the system learning how to improve itself. The loop does not merely search over task agents; it can also search over the improver that conducts the search. If the meta-agent learns better improvement procedures, then improvements should compound across generations and transfer across domains.

This paper tests that interpretation. We do not dispute that DGM-H-style systems work. They clearly do. They patch task agents, improve scores, and transfer useful scaffolding across domains. The question is narrower: when the system improves, what level of the system is doing the work? In particular, does continued meta-level self-modification produce content that goes beyond what the frozen base model would have proposed if asked once?

Our answer is: not in the artifacts studied here. The evidence supports a decomposition into two processes.

Task-level adaptation works. The loop exposes failures in the task agent, and the agent patches those failures. This is where evaluation earns its cost. The task prior cannot know the quirks of a particular benchmark, judge, reward function, parser, or domain until feedback reveals them. Iteration matters there.

Meta-level construction works once. Early in a run, the system elicits a general harness from the base model: analyze evaluations, track failures, revise prompts, use memory, budget calls, verify outputs, and repair obvious errors. These are useful ideas. They are also standard agent-design ideas already present in the model prior. Once elicited, the harness can be frozen and reused.

Continued meta self-modification is redundant. After the general harness exists, allowing the meta-agent to keep editing itself does not improve performance at matched budget, does not produce clearly novel meta-level mechanisms, and does not make the content distinguishable from one-shot frozen-model proposals.

The mechanism is simple. A scalar score can select among sampled variants, but it gives little attribution to the meta-level choices that caused future task-level outcomes. The meta-agent edits code; that code changes later patches; those patches change task outputs; the evaluator returns one number. That channel is strong enough to reject bad variants. It is weak as a signal for learning how the improver should change. When the generator cannot tell what to change, it defaults to what it already believes. For modern LLMs, that prior includes a substantial amount of general agent-scaffolding knowledge.

We support this with three lines of evidence.

First, we analyze the published DGM-H artifacts. Meta-edits are rare in nine of ten runs, generic in content, and often selected away. In the authors' own held-out transfer ablation, a frozen evolved meta-agent matches a continuing self-editing meta-agent at 50 generations. A wiring audit of the reconstructed final repositories shows that most showcased meta-level modules are present but unused by executing code.

Second, we run a pre-registered one-shot prior baseline. We compare loop interventions against proposals from the same frozen model given the generation-1 information set. A blinded discriminator separates task-level loop edits from one-shot prior samples, but it cannot separate meta-level loop edits from one-shot prior samples. This is the cleanest result in the paper.

Third, we extend the paradigm to philosophy writing under LLM-judged reward. This section is intentionally demoted: it is exploratory, lower-powered, and subjective. We include it because it is a useful stress test and because it reproduces the same signatures in a fourth domain: early task-level improvement, convergent harness motifs, unstable prompt/model elicitation, and no benefit

from forced meta-editing.

The target is not DGM-H as a system. The target is a mechanism claim. A loop can be useful without demonstrating continuing recursive self-improvement. Selection can filter variants without improving the selector. Transfer can show that a harness is reusable without showing that evolutionary feedback taught the harness anything domain-general.

Our contributions are:

1. a systematic secondary analysis of the published DGM-H artifacts, including transfer, lineage, and wiring analyses;
2. a low-cost one-shot-prior baseline for testing whether evolved meta content exceeds the frozen model prior;
3. a demoted exploratory extension to subjective philosophy writing, used as a stress test rather than a main proof;
4. a falsifiable reframing of patch-based self-improvement as one-time harness elicitation plus task-level adaptation.

2 Background and Setup

2.1 DGM-H / HyperAgents

In DGM-H/HyperAgents, each agent is a repository containing a task agent, a meta-agent, and shared utilities. At each generation, a parent is selected from an archive. The meta-agent receives the repository and evaluation artifacts, then patches any part of the codebase. The patched child is evaluated. If it performs well enough, it enters the archive and can become a parent for later generations.

The system is open-ended in the relevant sense: the edit surface includes the improver. The meta-agent can modify the task agent, its prompts, utilities, evaluation-analysis code, and its own future instructions. This is what makes the system appear to instantiate recursive self-improvement rather than ordinary search over task agents.

The published artifacts analyzed here include five main runs over paper review and Genesis robot-walking co-evaluation, five runs over IMO proof writing, and transfer ablations in which evolved agents are moved to a held-out `imo_grading` domain. Some transfer runs allow continued self-editing. Others freeze the evolved meta-agent.

The distinction between these conditions matters. A frozen evolved meta-agent tests whether a previously constructed harness is useful. A continuing self-editing meta-agent tests whether additional meta editability adds value on the new domain. If the latter does not beat the former at matched budget, then the transfer result alone cannot be evidence of ongoing self-improvement.

2.2 Philosophy-writing extension

We also implemented a philosophy-writing version of the loop. The task is to write complete short analytical philosophy papers under LLM-judged reward. The initial task agent is a simple planner-actor pipeline. A planner produces an argument plan. An actor then works through draft-editing tools and literature search for a bounded number of turns. Evaluation uses reviewer-style scalar

scores on visible training topics and separate blind validation topics whose artifacts are not shown to the meta-agent.

This section is included for two reasons. First, it tests whether the signatures observed in the published artifacts persist in a subjective, non-verifiable reward domain. Second, it lets us probe variants the original artifacts do not contain, including a forced dual-surface condition in which every patch must change both the writing pipeline and the meta/search machinery.

The philosophy results are not used as the main evidential base. They are n=1-per-configuration except for one rerun, use LLM judges, and are inherently noisier than the artifact analyses. They should be read as corroborative. Where they agree with the stronger evidence, they increase confidence in the mechanism. Where exact scores fluctuate, we do not treat them as decisive.

3 Mechanism: Selection Can Reject Without Teaching

The paper's core claim is not that scalar selection is useless. It is that scalar selection plays a different role from self-improvement.

A scalar score can support selection. It can say that one child is better than another, or that a broken child should not be retained. In patch-based agents, this matters. LLM-generated patches are unreliable. Some are empty, syntactically broken, note-only, or locally plausible but globally harmful. A score-informed archive filters that stream. Removing selection should hurt, and in the original results it does.

But improving the improver requires a different signal. The system would need to know which meta-level choices caused later improvements: whether a memory tool helped, whether an evaluation digest changed the next patch, whether a parent-selection heuristic increased future improvement rate, or whether a prompt expansion merely added words. A single task score per generation does not provide that attribution. It is separated from meta choices by at least two levels: the meta-agent changes code; changed code affects later patches; those patches affect task outputs; task outputs affect the score.

This distinction explains the observed split. At the task level, feedback is often localized. Judge prose, parser failures, crashes, reward-code exceptions, or accuracy breakdowns point to concrete fixes. The task agent can adapt to domain-specific failures. At the meta level, the feedback is diffuse. The improver does not receive a critique of its improvement strategy as an improvement strategy. It receives downstream task scores.

When a frozen LLM is asked to modify an agent without attributable feedback, its proposals come from its prior over agent design. That prior is not empty. It contains planning, critique, revision, evaluation summaries, memory, budget awareness, verification, prompt discipline, and multi-candidate selection. These are useful and general. They also explain why evolved meta-agents transfer: the content was general from the start.

This mechanism predicts several empirical patterns:

- meta-edits should converge on the same generic information-management motifs across domains;
- one-shot frozen-model samples should cover most loop meta-edits;
- task edits should move away from the one-shot prior as feedback accumulates;
- freezing an already constructed meta-harness should match continued self-editing at

matched budget;
- forcing the system to edit the meta level should produce generic machinery, not necessarily better improvement;
- richer meta-level feedback, such as patch critiques or attribution of outcomes to improvement choices, should help more than simply running longer with scalar selection.

The rest of the paper tests these predictions.

4 Evidence I: Published DGM-H Artifacts

4.1 Meta-edits are rare and generic

Across the published main runs, most patches target the task agent rather than the meta-agent. In nine of ten runs, `meta_agent.py` is touched in at most 7% of generations. The corresponding task-agent edit rate is roughly 78% to 98% in the paper-review/robotics runs and about 88% to 92% in the IMO-proof suite.

The exception is `exps/hyp_3`, where `meta_agent.py` is touched in 27 of 100 generations. This looks at first like evidence against rarity. On inspection, it is more consistent with the mechanism above. The edits occur inside instruction-string construction. They do not alter the single LLM call, candidate generation, parent selection, archive logic, or verification procedure. The run began with a minimal one-line meta-agent; several early children expanded it; the surviving expansion then prompted descendants to keep revisiting the meta-agent, producing repeated prompt redecorations. Many such edits became lineage dead ends, and the phase-controlled scores of meta-editing generations were not better than non-meta-editing generations.

The dominant meta-edit content is generic: analyze evaluations, identify bottlenecks, improve prompts, track memory, include remaining iterations, verify outputs, or summarize performance. These are sensible changes. They are also the standard vocabulary of agent scaffolding. The IMO-proof runs do not develop math-specific search operators inside the improver. The paper-review/robotics runs do not develop domain-specific meta machinery. Even the outlier run mostly edits how the LLM is instructed to think about improvement.

This matters because self-modification was available at every generation. The system could have edited its search procedure, patch-generation process, archive policy, or internal experimental design. Mostly, it did not. It patched task agents.

4.2 Surviving self-edits canalize rather than broaden search

Lineage analysis sharpens the point. In `exps/hyp_1`, the self-edits at generations 3 and 12 had no children and were selected away. The generation-1 self-edit, by contrast, became the ancestor of 97 of 100 generations. That edit expanded the meta prompt and named task-agent prompting as a priority.

The surviving self-edit therefore did not produce a richer meta-search process. It wrote the model's general prior into the harness and steered the rest of the run toward local task-agent prompt modifications. In this case, self-modification made prior-typical behavior structural.

That is still useful. A better prompt can make later task patches more directed. But it is not evidence

that scalar evolutionary feedback taught the improver a new improvement method. It is evidence that the system elicited a better initial harness and then used it.

4.3 The frozen-meta transfer ablation is the decisive comparison

The authors' transfer artifacts include a held-out `imo_grading` domain. At matched budget, the condition with continued self-editing reaches mean best accuracy 0.680 within 50 generations. The condition with the evolved meta-agent frozen reaches 0.688. Per-run best accuracies are similar: continuing self-edit gives 0.66, 0.63, 0.65, 0.80, 0.66; frozen meta gives 0.80, 0.64, 0.66, 0.67, 0.67.

Condition (held-out `imo_grading`)	Run 1	Run 2	Run 3	Run 4	Run 5	Mean
Continuing self-edit	0.66	0.63	0.65	0.80	0.66	0.680
Frozen evolved meta	0.80	0.64	0.66	0.67	0.67	**0.688**

Table 1: Best accuracy within 50 generations at matched budget, from the authors' transfer artifacts. Freezing the evolved meta-agent does not hurt.

This is the cleanest available ablation because it separates the value of having the evolved harness from the value of continuing to edit it. The evolved harness transfers. Continued editability does not add value at the same budget. Letting the continuing condition run much longer raises the mean best score, but the early matched comparison shows that most of the value comes from task-level adaptation under an already useful harness.

This result does not by itself prove that the harness came from the prior. A genuinely learned meta-agent could also transfer. But it does rule out a simple claim that ongoing meta editability is load-bearing on the held-out domain. To distinguish learned meta content from prior content, we need content analysis and one-shot baselines. Those are the next two pieces of evidence.

4.4 The wiring audit: much meta machinery is not executed

The original paper showcases meta-level additions such as performance trackers, memory systems, and evaluation analyzers. We audited whether these additions are wired into the agents that actually run.

For each of the ten published main runs, we reconstructed the dominant-lineage final repository by replaying the patch chain. We then classified meta-authored modules as live if they were reachable from the executing `meta_agent.py` or `task_agent.py`, or, for tools, if they defined the registration interface used by the framework. No dynamic imports appeared in the final agents, so the static import graph was sufficient.

The result is narrow. Across all ten final agents, live meta-level machinery consists of evaluation-result digests in three runs and a memory tool in one. These additions digest task performance for later task repair. They do not change candidate generation, selection, archive logic, or the outer loop.

Most other meta-looking modules are present but unwired. Examples include performance analyzers, prompt-template utilities, code-analysis helpers, response parsers, and prompt-engineering modules that are not imported by executing code. Some even document themselves as integrated while being unreachable in the import graph.

This pattern is important. It is not that the model never writes plausible meta machinery. It does. But absent attribution and integration pressure, much of it remains capability-shaped repository content rather than live improvement machinery. Where meta machinery is live, it mainly improves the information channel for task-level patching.

4.5 Re-reading the original ablations

The original results are often read as evidence for self-improvement because removing self-improvement hurts, removing open-ended exploration hurts, transferred meta-improvements help, and editing parent selection is possible. These observations are real. The question is what they isolate.

The relevant variables are: instruction content, task-level search, and meta editability. Many ablations vary more than one at a time.

When the initial one-line meta-agent is frozen from the start, performance is poor. That shows that freezing the wrong content is bad. It does not show that continued self-editing is necessary after the useful harness has been elicited. The transfer ablation freezes the evolved harness instead, and there freezing matches continued self-editing.

When open-ended archive search is removed, performance collapses. That supports the importance of task-level search and selection. It does not show that the meta-agent must keep modifying itself. Indeed, archive-on with frozen evolved meta works, while archive-off with editable meta fails.

When the evolved meta-agent improves transfer at `imp@50`, the measurement itself uses frozen metas. Both arms have editability off during the comparison. The result shows that starting from the better harness matters. It does not isolate the value of ongoing editability.

When the meta-agent is allowed to edit parent-selection mechanisms, it rediscovers standard strategies such as UCB-like selection, softmax sampling, and stagnation detection, but does not beat the handcrafted mechanism. This is exactly what the prior-default mechanism predicts: the model can produce textbook search machinery on demand, but producing it is not the same as learning a better improver through scalar feedback.

The original results therefore support a narrower interpretation. Task-level search is load-bearing. Harness content is load-bearing. Ongoing meta editability is not isolated as load-bearing in the available comparisons.

5 Evidence II: One-Shot Prior Baseline

The strongest test of the prior-elicitation hypothesis is direct. If evolved meta-edits are mostly the frozen model's prior, then one-shot samples from the same model, given the same early information, should cover them. If the loop teaches the improver something new, loop meta-edits should be distinguishable from one-shot proposals.

5.1 Design

We pre-registered a low-cost comparison before data collection. The loop corpus contains 120 sampled patches from the published DGM-H artifacts, stratified across the main domain suites

and generations. Notes-only patches were excluded. A blinded extractor converted each diff into mechanism-level intervention statements tagged as task, harness, or meta.

The one-shot corpus was generated by the same frozen model used as the original meta-agent. It received the generation-1 information set: the initial repository and initial evaluation report. Under the same minimal instruction, it produced concrete proposed diffs. The same extractor converted those proposals into intervention statements.

The primary endpoint avoids a hand-built motif taxonomy. A blinded judge, from a different model family than the generator, saw provenance-stripped intervention statements and estimated whether each came from the loop or from one-shot sampling. We report AUC overall and by level. Secondary endpoints include coverage@k: whether a loop intervention has a same-mechanism match among the first k one-shot samples.

5.2 Results

The corpora differ overall. The loop corpus has 826 statements; the one-shot corpus has 511. In the loop, 77% of statements are task-level, 15% harness-level, and 8% meta-level. In the one-shot samples, 59% are task-level, 4% harness-level, and 37% meta-level.

That composition is already informative. The frozen model eagerly proposes meta-level improvements when asked once. The loop rarely implements them. The proposal side is not the bottleneck.

The discriminator separates the corpora overall: AUC is 0.687. But the signal is concentrated at the task level. Task-level AUC is 0.717. Meta-level AUC is 0.465, effectively chance. Harness-level AUC is 0.624, based on a smaller and more imbalanced slice.

	Overall	Task	Harness	Meta
Share of loop statements (n = 826)	—	77%	15%	8%
Share of one-shot statements (n = 511)	—	59%	4%	37%
Discriminator AUC (loop vs. one-shot)	0.687	0.717	0.624	**0.465**
Coverage@30 by one-shot samples	0.521	—	—	**0.864**

Table 2: One-shot prior baseline. Task-level loop edits are distinguishable from one-shot frozen-model proposals; meta-level edits are at chance and mostly covered.

Coverage tells the same story. For all interventions, coverage@30 is 0.521. For improving generations, it is 0.504. For late generations, it is 0.432. For meta-level interventions, coverage@30 is 0.864 and has mostly saturated by k=20.

In plain terms: one-shot samples from the frozen model cover most loop meta-edits, but not most task edits. The loop's feedback advantage appears at the task level, not the meta level.

A matched-information subset confirms the direction. Restricting loop patches to the first few generations lowers overall distinguishability. Task-level edits become more distinguishable as feedback accumulates. Meta-level edits remain near chance throughout. This is exactly the pattern expected if task-level content learns from feedback while meta-level content stays near the prior.

5.3 Interpretation

The one-shot baseline is the paper's central content result. It does not merely show that evolved meta-edits are generic by human inspection. It shows that, under a blinded taxonomy-free endpoint, the loop's meta-level interventions are statistically indistinguishable from one-shot frozen-model proposals.

This does not mean the meta-level content has no value. Some of it is useful. A structured meta prompt, evaluation digest, or memory tool can make task patching easier. The point is about source and dynamics. The loop does not appear to discover meta content that the model could not already propose. It selects, freezes, and reuses prior-sourced harness content.

The contrast with task edits is crucial. The same methodology finds that task-level edits do move away from one-shot prior samples. That rules out a trivial explanation that the test is too weak to detect differences. It detects the differences where the mechanism says it should: at the level where feedback contains local task information.

6 Corroborative Extension: Philosophy Writing Under Subjective Reward

This section is intentionally demoted. The philosophy-writing experiments are not the main proof. They are lower-powered, use subjective LLM judges, and include fewer seeds. We include them because they stress the mechanism in a different domain and because they reproduce the same qualitative signatures.

The domain is also useful because it is favorable to the opposing hypothesis in one respect. Philosophy writing has no crisp unit tests and no parser errors. If open-ended self-improvement were going to invent richer meta procedures to handle ambiguous reward, this is a plausible place to see it. Instead, the same split appears: task-level scaffolds improve early; meta-level editing remains generic or unhelpful.

6.1 Early task-level gains, no sustained monotonic improvement

Across structured-prompt runs, the philosophy loop produces real early gains. A faithful 40-generation run improves from about 3.54 to about 4.15 on visible scores. Thirty-generation runs show similar visible improvements, and one varied-topic run improves on blind held-out scoring as well. A direct head-to-head later confirms that several evolved task harnesses beat the hand-built generation-0 agent under blind judging when using the same weak writer model.

But the best generation is not the final generation in any run. Peaks occur early, often between generations 3 and 15, followed by plateau or regression. This pattern has several possible contributors: noisy judges, winner's curse from selecting the maximum score, partial overfitting to visible topics, and genuine loss of useful structure in later patches. We do not overinterpret the exact magnitudes. The stable point is narrower: these runs do not show sustained recursive improvement. They show early task-level scaffold gains followed by instability.

6.2 Forced meta-editing does not help

The dual-surface condition required every patch to change both the writing pipeline and the meta/search machinery. The system complied. It edited `meta_agent.py` in 18 of 25 patch generations and parent-selection machinery in 16 of 25.

The content was again generic: archive summaries, diversity pressure, weakness-aware parent selection, context collection, and prompt improvements. These are sensible ideas. They did not improve the trajectory relative to regular task-focused runs. The dual-surface run peaked around the same visible score and then regressed. Conditions that prioritized meta-level editing performed worse.

This matters because it removes the easy objection that the system simply never tried to self-edit. When self-editing was forced, the system produced the expected prior-sourced machinery, but not better improvement.

6.3 Elicitation sensitivity

The philosophy extension also exposes strong prompt and model sensitivity. Under a structured prompt with inlined evaluation context, the meta-agent produces directed patches across runs. Under an upstream-style bare instruction, behavior becomes erratic for the tested models. Some invocations produce no patch; many produce notes, README changes, or changelogs instead of substantive code changes; local replications produce real code patches, but often to irrelevant or unused parts of the repository.

This does not prove that the original system only works because of a prompt artifact. The original Claude-based runs did work under the bare prompt. The point is more modest and more damaging to strong self-improvement claims: the apparent capability is heavily mediated by elicitation. Same loop, same edit surface, different prompt/model conditions, different behavior. That is what we should expect if the loop amplifies a base-model prior rather than learning a robust improvement procedure.

6.4 Convergent harness motifs

The evolved philosophy task harnesses converge functionally even when they differ textually. Regular-meta runs tend to rebuild the same template: compare plans, draft, critique, revise, run heuristic audits, repair conditionally, and stress-test objections. Weak-start runs reconstruct this scaffold within one or two generations. They do not generally rebuild tool-based literature retrieval, persistent cross-paper memory, or domain-specific meta-search.

This mirrors the published artifacts. The prior is good at reconstructing prompt-level and control-flow scaffolds. It is weaker at integrating tools and building persistent machinery unless the feedback channel makes those needs explicit.

6.5 Head-to-head and writer-swap

A blinded head-to-head compares fresh essays from the hand-built generation-0 agent against reconstructed best-generation evolved harnesses. Under the original weak writer setting, evolved

harnesses win decisively, including under a held-out judge. This supports the claim that task-level gains are real and not merely artifacts of the loop's visible judge.

A writer-swap pilot changes the writer model. Under the stronger/different writer, the evolved advantage disappears. This is consistent with the broader mechanism: evolved task scaffolds adapt to the operating point at which they were selected. They are useful, but not universal design knowledge. Their value can shrink or vanish when the base model changes.

6.6 What the philosophy section adds

The philosophy experiments do not carry the main claims. They add three pieces of texture.

First, subjective reward does not automatically produce meta-level improvement. Second, forcing meta-editing is not enough; without attribution, the system generates generic machinery. Third, the loop is highly sensitive to prompt and model elicitation, which is exactly what we should expect from prior amplification.

The blunt version: the philosophy section is interesting, but it is not as strong as the artifact analysis or one-shot baseline. It belongs in the paper as a demoted corroborative section, not as an equal third pillar.

7 Interpretation: One-Time Harness Search Plus Task Adaptation

The evidence points to a two-stage account.

In the first stage, the system elicits a general improvement harness from the base model. This harness says to inspect evaluations, identify bottlenecks, improve prompts, track past failures, use remaining budget, and verify outputs. The harness is useful because modern LLMs already contain a lot of agent-design knowledge. It can transfer because it is domain-general.

In the second stage, the frozen or effectively frozen harness conducts task-level patching. This is where the loop's evaluation budget pays off. Task feedback reveals benchmark quirks, domain failures, parser problems, bad assumptions, and judge preferences. Selection filters harmful patches and preserves useful ones.

What is missing is a third stage: continuing improvement of the improver. In the available artifacts, meta self-edits do not accumulate into better search procedures. They mostly expand prompts, summarize evaluations, or create generic modules. Some become dead code. Some help task patching by improving the information channel. But they do not make the meta-agent visibly better at generating future improvements.

This interpretation also explains why selection matters. The loop is not merely random sampling. Selection rejects bad variants and retains useful task adaptations. Removing it should and does hurt. But that does not imply the generator has learned. A best-of-N process can be useful while still drawing from a fixed proposal distribution. The empirical discriminator is whether freezing the proposal mechanism hurts. In the transfer ablation, freezing the evolved meta does not hurt at matched budget.

The same distinction applies to transfer. A transferred frozen harness can be valuable for two reasons. It might encode improvement knowledge learned from source-domain evolutionary feedback. Or it might encode domain-general prior knowledge that was elicited once and then reused.

Transfer alone cannot separate these. Content can. The one-shot baseline and wiring audit favor the prior-sourced reading.

8 Predictions and Falsifiability

The claim is falsifiable. Several results would weaken it.

A vanilla scalar-selected loop could produce meta-level content that one-shot prior samples do not cover, while task-level and meta-level comparisons are controlled for information set. That would show that the meta layer is moving beyond the prior.

A continuing self-editing meta-agent could beat a frozen evolved meta-agent at matched budget across fresh domains and seeds, with the same task-level archive and evaluation budget. That would isolate ongoing editability as load-bearing.

A wiring audit could show that evolved meta machinery changes candidate generation, patch verification, parent selection, or internal experimentation, and that ablations of those components reduce later improvement rate. That would demonstrate live meta-level mechanisms rather than repository artifacts.

Conversely, the mechanism predicts that simply running longer with scalar feedback will not reliably produce better meta-improvers. The interventions most likely to help are those that enrich the meta-level feedback channel: qualitative critiques of patches as patches, attribution of outcome changes to meta choices, explicit selection on improvement ability rather than raw task score, and persistent lessons that the meta-agent actually reads and updates.

It also predicts a bitter-lesson-style trend. As base models improve, one-shot coverage of evolved meta-edits should rise, because more scaffold knowledge will be in the prior. Meta-editability should become less useful unless the feedback channel becomes richer.

9 Limitations

The analysis is limited to the published artifacts and to domains with the general shape "LLM produces artifact, evaluator scores artifact." The results may not transfer to embodied control, long-horizon interaction, multi-agent environments, or domains where meta choices receive richer feedback.

The transfer ablation has only five runs per condition. It is still unusually informative because it cleanly compares frozen evolved meta against continuing self-editing at matched budget, but the sample size is small.

The one-shot baseline depends on extraction and matching judgments. We reduce circularity by using a taxonomy-free discriminator as the primary endpoint, but the extracted intervention statements are still lossy. This likely biases coverage downward, not upward, but it remains a limitation. The meta-level sample is also smaller than the task-level sample because loop meta-edits are rare.

The philosophy extension is exploratory. It uses subjective LLM judges, limited seeds, and a researcher-built starting harness. Exact scores should not be treated as stable. The section is included for qualitative convergence and stress testing, not as a decisive empirical proof.

The wiring audit classifies static reachability. If a module is dynamically used in a way not visible to imports or tool registration, the audit could miss it. In the reconstructed final agents, no dynamic

imports were observed, so this risk appears low.

Finally, the paper does not show that meta-level self-improvement is impossible. That would be too strong. It shows that in these artifacts, under scalar-selected patch loops at these budgets, the observed meta-level content is better explained as prior-sourced harness construction than as continuing recursive improvement.

10 Related Work

The Gödel machine is the theoretical ancestor of provably self-improving systems. DGM replaces proof with empirical validation and open-ended archive search. DGM-H/HyperAgents extends the idea to multi-domain agents whose meta-agent can patch the entire repository. This paper analyzes that setting directly.

STOP and related scaffold-improvement systems already suggested that frozen language models tend to propose recognizable standard techniques. Our contribution is to make that observation systematic: the same generic meta motifs recur across domains, and one-shot samples cover most loop meta-edits.

Work on coding agents such as SWE-agent, OpenHands, SICA, and related systems shows that frozen or hand-designed scaffolds can drive substantial task-level improvement. This is consistent with our decomposition. The scaffold can be fixed while the artifact being improved changes. In coding domains, tracebacks and tests provide unusually rich localized feedback, which helps task-level repair.

Work on sampling baselines for scaffold evolution is especially relevant. Budget-matched IID or sequential sampling can often compete with evolutionary scaffold search once noisy validation is accounted for. That result is complementary to ours: sampling baselines compare performance; our one-shot baseline compares content.

Textual-gradient and reflection systems such as TextGrad, GEPA, and reward-design systems such as Eureka clarify the boundary condition. Iteration works best when feedback is diagnostic. A critique that says why a patch failed can steer the next patch. A scalar score, especially one separated from meta choices by multiple steps, mostly selects.

The Huxley-Gödel Machine's emphasis on metaproductivity is closely aligned with our diagnosis. If the goal is to improve the improver, the selection target should be improvement ability, not raw downstream task score. Without that distinction, a system can get better task agents without learning a better meta-agent.

11 Conclusion

DGM-H-style systems are useful. They improve task agents, transfer general scaffolds, and exploit selection to filter unreliable patches. The mistake is to call all of that self-improvement in the strong recursive sense.

The artifacts studied here support a narrower interpretation. The meta level contributes a useful one-time harness, mostly drawn from the base model prior. Once that harness exists, freezing it does not hurt at matched budget. Continued meta self-modification is rare, generic, often unwired, and indistinguishable from one-shot prior samples. The task level, by contrast, does move beyond

the prior as feedback accumulates.

Selection is therefore doing real work, but not the advertised work. It validates and filters. It does not, by itself, teach the improver how to improve. Future claims of recursive self-improvement should include frozen-framework baselines, one-shot-prior baselines, and audits of whether evolved meta machinery is actually executed. Without those controls, transfer and improvement results remain compatible with a simpler explanation: one-time prior elicitation plus task-level adaptation.